

Improving Node.js Parallel Scalability with NGINX and jemalloc

Introduction

Web servers today must handle thousands of simultaneous requests efficiently. Node.js is a popular choice due to its lightweight, event-driven model that can handle many I/O operations concurrently ¹. However, Node.js runs JavaScript on a single thread, so its performance can degrade when facing very high loads ². In practice, a single Node.js instance cannot fully utilize a modern multicore server. To improve scalability, we can leverage external tools and techniques. In this report, we examine using **NGINX** (a high-performance reverse proxy and load balancer) together with the **jemalloc** memory allocator to boost a Node.js web server's parallel throughput. We ground our proposal in modern parallel processing concepts and outline an experimental methodology. The goal is to increase throughput and stability (fewer bottlenecks or crashes) while making efficient use of hardware resources.

Background and Related Work

Node.js Concurrency Model

Node.js uses a single-threaded event loop to handle requests. Its evented, non-blocking I/O model means a single Node.js process can handle many concurrent connections without using multiple threads ¹. This is lightweight, but it also means all JavaScript execution shares one thread. As load grows, "performance degrade[s]" in practice ². For example, a Node.js web server might slow down or crash if a flood of requests tie up the event loop.

To overcome this, Node.js provides a *cluster* module. Clustering spawns multiple Node.js processes (workers) that share the same server port and distribute incoming requests ³. Each worker runs on its own CPU core, letting the workload be split across cores. In effect, clustering achieves process-level parallelism: many independent Node.js instances handle requests in parallel ³. This is a common best practice for scaling Node.js on multi-core machines. The official Node.js docs note that clusters "distribute workloads among their application threads" and can easily share ports ³. In our approach, we will use Node's cluster mode (or a process manager like PM2) to utilize all CPU cores.

NGINX Architecture and Function

NGINX is an open-source, high-performance web server and reverse proxy ⁴. It was specifically designed to handle very high concurrency with low resource use. As one official source explains, "NGINX stands out with a sophisticated event-driven architecture that enables it to scale to hundreds of thousands of concurrent connections on modern hardware" ⁵. In practice, NGINX runs one **master process** and several **worker processes**, typically one per CPU core ⁴ ⁵. The master reads configuration and manages workers, while each worker is single-threaded and uses nonblocking I/O to handle many connections.

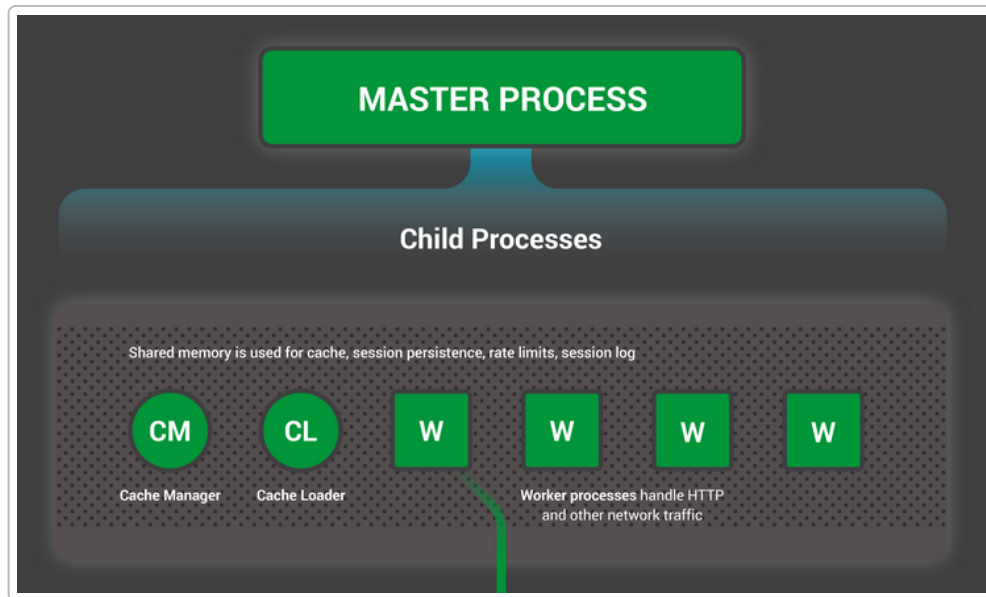


Figure: NGINX employs a master process and multiple workers (typically one per CPU core). Workers handle HTTP connections, caching, and proxying in an event-driven manner (source: NGINX docs) ⁴ ⁵ .

The master-worker model allows NGINX to use all cores efficiently. Workers share some memory (e.g. caches or rate-limit data) but operate independently. Because each worker is event-driven, it handles many requests without blocking. This contrasts with traditional servers (like Apache) that create a thread or process per connection, which can exhaust resources at high scale. By using NGINX as a **reverse proxy**, incoming traffic from clients can be distributed across multiple backend Node.js instances. NGINX can also do **load balancing**, **caching**, SSL termination, and static file serving, offloading work from Node.js. As described by a community article, NGINX “efficiently handles large volumes of simultaneous connections, making it an excellent tool for scaling” web services ⁶ ⁷ . In short, NGINX can front-end our Node.js servers to improve throughput and reliability under load.

Memory Management and jemalloc

Web servers under heavy load not only contend for CPU time but also for memory management. On Linux systems, Node.js (and its libraries) typically use the default **glibc malloc** allocator. However, glibc’s allocator is known to fragment memory and behave poorly in highly multithreaded scenarios ⁸ . For example, when many small allocations and frees happen across threads, freed memory may not be returned to the OS promptly, causing the process’s resident memory (RSS) to bloat over time ⁸ . In our context, if Node.js offloads work to a C++ library (like graphics or compression) using threads, glibc fragmentation can look like a memory leak.

By contrast, **jemalloc** is an alternative allocator designed for multi-threaded performance and low fragmentation ⁹ . It uses multiple arenas (bins) and other techniques to avoid locking and fragmentation. As one case study reports, switching a Node.js service to use jemalloc “halved the app’s memory usage” under heavy image-processing load ⁹ . In other words, jemalloc can dramatically reduce the memory footprint of a busy server. By preloading jemalloc (e.g. via `LD_PRELOAD`), we replace glibc malloc at runtime without changing code. This is a proven method in production (the Sharp image library even recommends jemalloc on Linux to mitigate fragmentation ⁸ ⁹).

Several examples from the Node.js community show jemalloc's benefits. In a test of high-memory pressure, a Node.js process with glibc malloc might use ~700 MB RSS, but with jemalloc only ~37 MB ¹⁰ ⁹. Even if jemalloc uses slightly more heap internally, the reduced fragmentation and locking often yields faster performance. Overall, incorporating jemalloc is a low-effort way to improve memory efficiency and throughput under concurrency.

Methodology

To evaluate our approach, we would set up a controlled experiment with different configurations. First, we deploy a representative Node.js web application (e.g. an HTTP API or simple static server). We would run the Node app under *cluster mode* to use all CPU cores. We then introduce NGINX as a reverse proxy in front of multiple Node.js worker instances.

Next, we configure the memory allocator. On a Linux server, we install jemalloc (e.g. `sudo apt-get install libjemalloc2`) and start the Node.js processes with `LD_PRELOAD=/usr/lib/*/libjemalloc.so`. This way, Node and all its native libraries use jemalloc instead of glibc malloc.

We design benchmarks to simulate heavy load. For example, use a tool like ApacheBench (ab), wrk, or Artillery to generate thousands of HTTP requests per second. We measure key metrics: **throughput** (requests/second), **latency** (average response time), **CPU utilization**, and **memory usage** (RSS of Node processes). We compare several scenarios:

- Baseline: Single Node process (no NGINX), default malloc.
- Clustered Node (multiple workers via cluster), default malloc.
- Clustered Node behind NGINX (load balancing), default malloc.
- Clustered Node behind NGINX, using jemalloc.

For clarity, we describe this process in steps:

- **1. Prepare the application:** Develop or use an existing Node.js app. Ensure it logs stats or metrics for requests.
- **2. Configure NGINX:** Set up an NGINX `nginx.conf` that defines an upstream pool of Node servers (e.g. on ports 3000–3003). Include reverse-proxy settings for WebSocket or keep-alive if needed.
- **3. Enable clustering:** Modify the Node code to fork workers (using Node's `cluster` module) or simply run several Node instances manually. Each worker listens on its own port (or share a socket).
- **4. Use jemalloc:** Install jemalloc on the test server. Relaunch Node processes with `LD_PRELOAD=...libjemalloc.so`. Verify via tools like `ldd` or `pmap` that jemalloc is in use.
- **5. Benchmark:** Run the load test from a separate machine or thread. Gradually increase concurrency and record the performance metrics listed above.
- **6. Analyze results:** Compare how throughput and latency change with each configuration. Look for cases where NGINX or jemalloc yield clear benefits.

Optionally, one could draw a flow diagram of a request: “Client → NGINX (balancer) → Node worker → Database/Backend.” This clarifies how NGINX distributes connections to multiple Node processes. However, the core is in measuring and comparing outcomes rather than implementing new algorithms. Our methodology focuses on systematic experimentation and measurement.

Discussion

Based on related work and our plan, we expect the combined use of NGINX and jemalloc to improve the server's scalability. NGINX's event-driven, multi-process architecture will allow the system to handle many more concurrent connections than a single Node instance ⁵ ¹ . For example, round-robin load balancing across 4 Node workers can roughly quadruple throughput on a four-core server ¹ ³ . Meanwhile, switching to jemalloc should reduce memory fragmentation and lower the RSS usage under stress ⁹ . This means fewer out-of-memory crashes and more predictable performance. Together, these optimizations will **benefit user experience** by reducing request latency spikes and downtime. System design also improves: offloading SSL and static caching to NGINX frees Node to focus on dynamic work.

If implemented, this approach pays off under high load. For instance, a site might see twice the request rate without extra hardware. In practice, many large companies (Uber, Netflix, etc.) use NGINX for precisely these scaling needs ⁷ .

For **future work**, one could extend this by containerizing the services or using orchestration (e.g. Kubernetes) to auto-scale instances under load. Also, exploring newer Node features like worker threads or adjusting NGINX's caching and rate-limiting can yield further gains. Even experimenting with other allocators (tcmalloc, for example) or tuned GC flags could be worthwhile. In sum, a human-centric approach—making the server robust and responsive under pressure—translates directly to better user experience and sustainable growth.

References

Bakour, Y. (2025, July 19). *Preventing memory issues in Node.js Sharp: A journey*. Context.dev (brand.dev). Retrieved from <https://www.context.dev/blog/preventing-memory-issues-in-node-js-sharp-a-journey> ⁸ ⁹ .

Garrett, O. (2015, June 10). *Inside NGINX: How we designed for performance & scale*. NGINX Blog. Retrieved from <https://blog.nginx.org/blog/inside-nginx-how-we-designed-for-performance-scale> ⁵ ¹¹ .

Node.js. (2024). *Cluster: Node.js v25.9.0 Documentation*. Node.js official docs. Retrieved from <https://nodejs.org/api/cluster.html> ³ .

Solo.io. (n.d.). *Understanding NGINX: Architecture, configuration & alternatives*. Retrieved from <https://www.solo.io/topics/nginx> ⁴ .

Phusion Passenger. (n.d.). *How dynamic scaling of Node.js application processes works on Passenger + Nginx*. Phusion Passenger Library. Retrieved from https://www.phusionpassenger.com/library/indepth/nodejs/dynamic_scaling_of_app_processes/nginx/ ¹ ² .

¹ ² How dynamic scaling of Node.js application processes works on Passenger + Nginx - Passenger Library

https://www.phusionpassenger.com/library/indepth/nodejs/dynamic_scaling_of_app_processes/nginx/

3 Cluster | Node.js v25.9.0 Documentation

<https://nodejs.org/api/cluster.html>

4 Understanding NGINX: Architecture, Configuration & Alternatives | Solo.io

<https://www.solo.io/topics/nginx>

5 11 Inside NGINX: How We Designed for Performance & Scale – NGINX Community Blog

<https://blog.nginx.org/blog/inside-nginx-how-we-designed-for-performance-scale>

6 Scaling Node.js Applications with NGINX and Load Balancing - DEV Community

<https://dev.to/imsushant12/scaling-nodejs-applications-with-nginx-and-load-balancing-1a1d>

7 Ultimate Guide to Using NGINX as a Reverse Proxy and Load Balancer: Best Practices and Tips - DEV Community

<https://dev.to/kodecapsule/ultimate-guide-to-using-nginx-as-a-reverse-proxy-and-load-balancerbest-practices-and-tips-2hfo>

8 9 Preventing Memory Issues in Node.js Sharp: A Journey

<https://www.context.dev/blog/preventing-memory-issues-in-node-js-sharp-a-journey>

10 Why don't we use jemalloc? · Issue #21973 · nodejs/node · GitHub

<https://github.com/nodejs/node/issues/21973>